Optimization-AI

Plain-English in → provably-optimal decision out. An LLM front-end over a real operations-research solver.

What it is, in one sentence

You describe a logistics or planning problem in ordinary language; the system figures out *what kind* of optimization problem it is, pulls the numbers out of the prose, runs an exact mathematical solver, and explains the answer back in plain English — with no spreadsheet, no modelling language, and no solver expertise required from the user.

Why this is worth something

Operations Research already *solves* these problems optimally — the hard part has always been the human bottleneck in between: someone has to translate a messy business question into a precise mathematical model. That step needs a specialist and takes hours. This system collapses that step to a sentence, while keeping the answer **exact** (it is a real solver, not the LLM guessing numbers).

What it can do today — two live examples

Example 1 Classic transportation (linear program)

> *"Two factories produce widgets: Seattle has capacity 350 and San Diego has capacity 600. Ship to New York (325), Chicago (300), Topeka (275). Distances in thousand miles … freight is $90 per case per thousand miles. Minimize total shipping cost."*

Result: optimal shipping plan, total cost **$153,675** — matches the textbook optimum exactly.

Example 2 Fixed-charge logistics (mixed-integer program)

> *"Ship from 2 plants to 3 customers… on top of per-unit cost, opening any route costs a fixed $500 regardless of volume. Minimize total cost including the fixed charges."*

Result: optimal cost **1660** — the solver decides it is cheaper to consolidate onto **three** routes

(P1→M1, P1→M2, P2→M3) and leave the rest closed. This is a genuinely hard combinatorial decision (which routes to open), not just arithmetic.

How it works

Every stage is a discrete, inspectable step — not one opaque prompt:

```
natural language
   -> Intent router      (new problem / follow-up / chit-chat)
   -> Classifier         (transportation? scheduling? — majority vote, schema-bound)
   -> Specialist extractor (prose -> typed parameter table)
   -> Feasibility check  (3 layers: structural -> domain rules -> LP relaxation)
   -> Solver             (Pyomo + HiGHS — exact LP / MIP)
   -> Explanation        (plain-language summary, numbers grounded in the solution)
```

The feasibility gate matters: if the LLM hallucinates a number, the structured checks catch the inconsistency *before* the solver runs, so a bad extraction fails loudly instead of producing a confident wrong answer.

What is solid vs. honest limitations

Solid today	Honest limitations
• Transportation (LP) & fixed-charge (MIP) — exact, verified against textbook optima • Single-stage scheduling • Deterministic numeric output (same answer every run, any machine) • Runs fully local & private (Ollama / qwen3:14b) — no data leaves the box • Round-trip eval harness: 3/3 pass, recall 1.0	• First answer is slow on CPU (~2–3 min of local LLM inference) — the next roadmap item makes the wait visible/streamed • Two problem families so far; more to add • Heuristic warm-start helps LP, not yet fixed-charge-aware • Explanation wording varies slightly run-to-run (the *numbers* do not)

The high-level picture ends here. The rest of this document is for anyone who wants to dive into *how* it actually works, the modelling assumptions, and what is next.

Part II — Technical deep dive

For the reader who wants the internals.

1. How a request is orchestrated

A single entry point (`agent/core.py → solve_natural_language`) runs a fixed, inspectable pipeline. Every stage is a discrete step with its own module, not one opaque megaprompt:

1. **Intent router** — classifies the message as *smalltalk / help / follow-up / optimization*. The first three short-circuit; only a genuine problem enters the solve pipeline.
2. **Problem classifier** — picks the problem family and the concrete `solver_id` using an *N-vote majority* over a JSON-schema-bound prompt. Low confidence or "UNKNOWN" stops here with a clear message rather than guessing.
3. **Specialist extractor** — a per-family extractor (`TransportationSpecialist` / `SchedulingSpecialist`) turns prose into a typed parameter dict, mirroring an example schema and omitting optional fields (e.g. `fixed_cost`) when not mentioned.
4. **Validation + 3-layer feasibility** — structural checks, then problem-specific necessary conditions (e.g. total supply ≥ total demand), then an LP-relaxation solve. An infeasible instance returns plain-language reasons *and concrete repair suggestions*, and is remembered so the next message can fix it.
5. **Mode routing** — `exact`, `heuristic`, or `heuristic_then_ask` (see §2).
6. **Solve → explain** — Pyomo + HiGHS produces the numbers; the LLM narrates them. The solution is stored in the conversation context so follow-ups and what-ifs work against it.

2. The two-call heuristic protocol

Local LLM inference costs ~2–3 min on CPU. Rather than make the user stare at a spinner, `heuristic_then_ask` mode splits the work:

- **Call 1:** a fast constructive heuristic (Vogel's Approximation for transport, Longest-Processing-Time for scheduling) returns a feasible answer immediately, together with an *LP-relaxation lower bound* → an honest "this is at most X% above optimal" gap. The job is persisted in an in-memory store (10-minute TTL).
- **The user decides:** `optimize` / `accept` / `use heuristic` — or just asks a free-text what-if, which is answered against the pending job without consuming it.

- **Call 2 (optimize):** the exact solver runs *warm-started* from the heuristic's solution (Pyomo `var.value=…` before solve; APPSI-HiGHS picks it up automatically) → a provably-optimal answer, reached faster.

3. How the optimization is actually solved

Models are built with **Pyomo** and solved by **HiGHS** through its `highspy` wheel via Pyomo's APPSI interface — no external solver binary to install (this is also why Windows setup is now trivial).

Transportation — linear program

```
min   Σ_ij  c_ij · x_ij
s.t.  Σ_j x_ij ≤ capacity_i        (supply)
      Σ_i x_ij ≥ demand_j          (demand)
      x_ij ≥ 0     (optional per-arc capacity x_ij ≤ u_ij)
```

Fixed-charge logistics — mixed-integer program

```
min   Σ_ij c_ij · x_ij  +  Σ_ij f_ij · y_ij
s.t.  transportation constraints, plus
      x_ij ≤ M_ij · y_ij ,   y_ij ∈ {0,1}
      M_ij = min(capacity_i, demand_j)   (tight big-M)
```

The binary `y_ij` = "is route (i,j) open?" turns this into a real combinatorial decision. Supplying a `fixed_cost` automatically switches the model to a MIP, enables the LP-relaxation bound, and trips the warm-start gate. Single-stage scheduling is modelled analogously (assignment + sequencing, makespan objective; LPT heuristic for the warm answer).

Determinism: HiGHS returns the exact optimum and classification/extraction run at temperature 0, so the objective and flows are byte-identical on any machine; only the narration wording can vary.

4. The LLM layer

An abstract `LLMClient` hides the provider. The default is local Ollama with `qwen3:14b` across all stages (one warm model); a Groq backend is opt-in. Per-stage temperature: classification 0 (schema-bound, voted), extraction 0, explanation 0.1–0.3 (wording only).

The key design rule: **the LLM never computes the optimum.** It only translates prose ↔ schema and narrates the solver's output. The 3-layer feasibility gate is the guardrail — a hallucinated parameter fails loudly *before* the solver runs, instead of producing a confident wrong number.

5. Follow-ups & what-if

Follow-ups are classified into *sensitivity / what-if / resolve / pareto*. A what-if re-parses just the changed parameter, re-checks feasibility, re-solves, and reports the cost delta and flow changes against the baseline. An infeasible scenario is treated as a useful answer ("that can't work, here's why, here's how to fix it"), not an error — and it works even while a heuristic job is still pending.

Assumptions & scope

Being explicit about what the current system assumes:

- The LLM extracts and explains; **all arithmetic is the solver's**.
- Transportation is single-commodity, single-period; costs are linear in flow; capacities, demands and costs are point estimates (no uncertainty/stochastics).
- A fixed charge, when present, is incurred once per opened route regardless of volume; `M_ij = min(capacity_i, demand_j)` is taken as a valid tight bound.
- The fast feasibility checks assume standard supply/demand structure; the LP-relaxation layer is the backstop for anything they miss.
- Scheduling is single-stage.
- Reproducibility assumes the same model (`qwen3:14b`) and solver (HiGHS); explanation prose may differ run to run.
- Single-process, in-memory job store with a 10-minute TTL — not yet horizontally scaled.
- Demo posture: trusted input, no authentication / rate-limiting / multi-tenancy.

Future work

- **Make the wait visible** — stream the classify/extract/solve pipeline live so the LLM latency reads as progress, not dead air.
- **Follow-up chips & graceful fallback** — one-click what-if / sensitivity on the last solution.
- **Spreadsheet / API fast path** — upload an Excel sheet (or post structured params) and skip both LLM calls entirely; Excel export of results.
- **Fixed-charge-aware heuristic** — the reported heuristic cost is now correct, but the warm start (VAM) is still fixed-charge-blind; a charge-aware constructive heuristic would tighten the gap.
- **More problem families** — multi-period / multi-commodity flow, richer scheduling, beyond the current two.
- **Evaluation depth** — extend the round-trip eval harness (Phase 3) and add adversarial

extraction tests.

- **Productionization** (if it graduates from demo) — persistent/scalable job store, auth, rate limiting.
